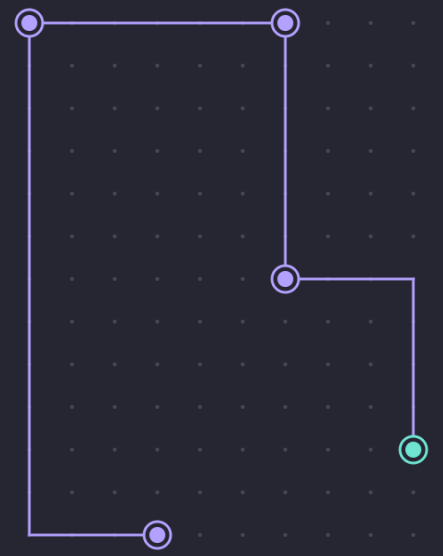
// COMMUNAUTÉ .NET

Rendre un test explicite et fiable, sans rien céder sur le modèle métier

Votre test est vert. Mais est-il compréhensible ? Et dit-il la vérité ?



Sylvain Aurat · Chapter Lead .NET


Qui ici écrit des mocks
dans ses tests ?


Et qui écrit des stubs, des fakes,
des spies ou des dummies
dans ses tests ?

Test doubles

Avant d’aller plus loin, mettons-nous d’accord sur les mots.



⚠ **Dans beaucoup d’équipes, « mock » sert de terme générique.**

Test doubles

Définitions de référence de Gerard Meszaros (*xUnit Test Patterns*).



CONTRÔLE

Stub

Retourne des réponses prédéfinies pour contrôler le scénario.

xunitpatterns.com

Test doubles

Définitions de référence de Gerard Meszaros (*xUnit Test Patterns*).



CONTRÔLE

Stub

Retourne des réponses prédéfinies pour contrôler le scénario.

SIMPLIFIE

Fake

Fournit une implémentation fonctionnelle, mais simplifiée.

[illegible]

xunitpatterns.com

Test doubles

Définitions de référence de Gerard Meszaros (*xUnit Test Patterns*).



CONTRÔLE

Stub

Retourne des réponses prédéfinies pour contrôler le scénario.

SIMPLIFIE

Fake

Fournit une implémentation fonctionnelle, mais simplifiée.

ENREGISTRE

Spy

Enregistre les interactions pour les vérifier après exécution.

[illegible]

[illegible]

xunitpatterns.com

Test doubles

Définitions de référence de Gerard Meszaros (*xUnit Test Patterns*).



CONTRÔLE

Stub

Retourne des réponses prédéfinies pour contrôler le scénario.

SIMPLIFIE

Fake

Fournit une implémentation fonctionnelle, mais simplifiée.

ENREGISTRE

Spy

Enregistre les interactions pour les vérifier après exécution.

ATTEND

Mock

Vérifie que les interactions attendues ont eu lieu.

[illegible]

xunitpatterns.com

Test doubles

Définitions de référence de Gerard Meszaros (*xUnit Test Patterns*).



CONTRÔLE

Stub

Retourne des réponses prédéfinies pour contrôler le scénario.

SIMPLIFIE

Fake

Fournit une implémentation fonctionnelle, mais simplifiée.

ENREGISTRE

Spy

Enregistre les interactions pour les vérifier après exécution.

ATTEND

Mock

Vérifie que les interactions attendues ont eu lieu.

REMPLIT

Dummy

Remplit un paramètre obligatoire, sans être utilisé.

xunitpatterns.com

Test doubles

Définitions de référence de Gerard Meszaros (*xUnit Test Patterns*).



CONTRÔLE

Stub

Retourne des réponses prédéfinies pour contrôler le scénario.

SIMPLIFIE

Fake

Fournit une implémentation fonctionnelle, mais simplifiée.

ENREGISTRE

Spy

Enregistre les interactions pour les vérifier après exécution.

ATTEND

Mock

Vérifie que les interactions attendues ont eu lieu.

REMPLIT

Dummy

Remplit un paramètre obligatoire, sans être utilisé.

xunitpatterns.com

Dummy

Un dummy est un argument nécessaire à l'appel, mais sans influence sur le comportement testé.

Définition élargie du *Dummy Object* de Gerard Meszaros (*xUnit Test Patterns*, 2007) : étendue à tout argument et centrée sur le comportement testé plutôt que sur son utilisation effective dans le code.

Le scénario testé en dépend-il ?

Scénario testé : un client professionnel obtient 15 % de remise.

PricingService.cs

```
public decimal ComputeDiscount(CustomerType type, int quantity, decimal orderTotal) {
    if (type == CustomerType.Business) {
        return orderTotal * 0.15m; // scénario testé
    }

    return quantity ≥ 10 ? 5m : 0m; // autre scénario
}
```

type Déterminant

Choisit la branche exécutée.

orderTotal Déterminant

Entre dans le calcul de la remise.

quantity Dummy

Exigé par la signature, sans effet sur ce scénario.

Un dummy respecte les règles, même inutilisé

On ajoute une garde : une quantité doit être strictement positive.

PricingService.cs

```
public decimal ComputeDiscount(CustomerType type, int quantity, decimal orderTotal) {
    if (quantity ≤ 0) { throw new ArgumentOutOfRangeException(nameof(quantity)); }

    if (type == CustomerType.Business) {
        return orderTotal * 0.15m; // scénario testé
    }

    return quantity ≥ 10 ? 5m : 0m; // autre scénario
}
```

⚠ **Inutilisé par le scénario ne veut pas dire jamais lu par le code.**

Un dummy se reconnaît à son (non) rôle

InvoiceTests.cs

```
[Fact]
public void ShouldSumInvoiceLines() {
    var invoice = new Invoice(
        InvoiceNumber.Parse("INV-2026-0042"), // Value Object
        Customer.Register("Alice Martin"),    // entité
        paymentTermDays: 30,                  // int
        note: "Livrer avant midi",            // string
        lines: [new InvoiceLine(quantity: 3, unitPrice: 20m)]);   // l'assertion ne dépend que de quantity et unitPrice

    Assert.Equal(60m, invoice.Total);
}
```

dummies

// 01 · VOCABULAIRE

Value patterns

Trois façons de fournir une valeur à un test.



D'où vient la valeur ?

FIGÉE

Literal Value

Écrite telle quelle dans le test, comme 240 km.

CALCULÉE

Derived Value

Obtenue à partir d'autres valeurs du test.

VARIABLE

Generated Value

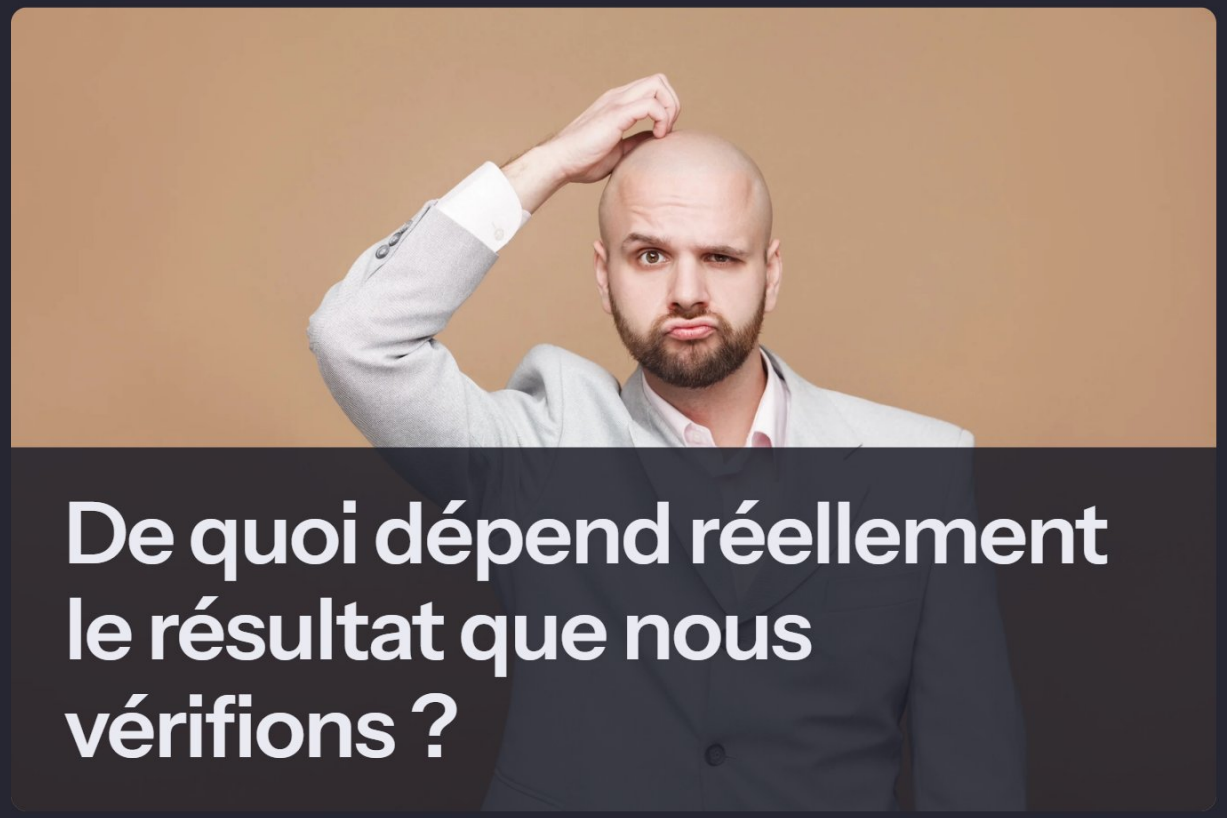
Produite à l'exécution, en séquence ou au hasard.

xunitpatterns.com

// 02 · LE PROBLÈME

Un test tout à fait ordinaire

Il compile, il passe, et son nom dit exactement ce qu'il vérifie, mais…



De quoi dépend réellement le résultat que nous vérifions ?

```
[Fact]
public void ShouldEstimateShipmentCO2Emissions() {
    var shipment = new Shipment(
        ShipmentId.Parse("SHP-2026-001872"),
        new Route(
            RouteId.Parse("RTE-8821"),
            Distance.FromKilometers(240.0),
            RouteType.Highway),
        new Parcel(
            ParcelId.Parse("PKG-00152"),
            Weight.FromKilograms(18.0),
            new Dimensions(
                Length.FromMeters(0.6),
                Length.FromMeters(0.4),
                Length.FromMeters(0.3)),
            ParcelCategory.Standard),
        new Vehicle(
            VehicleId.Parse("VEH-271"),
            FuelType.Diesel,
            FuelConsumption.FromLitersPer100Km(6.5),
            EnginePower.FromKilowatts(95.0)),
        new WeatherConditions(
            Temperature.FromCelsius(16.0),
            Humidity.FromPercent(60.0)),
        EmissionFactor.FromKilogramsPerLiter(2.31));

    var calculator = new EmissionCalculator();

    var result = calculator.Estimate(shipment);

    Assert.Equal(36.036, result, precision: 3);
}
```

// 02 · LE PROBLÈME

Un test vert, mais fragile

Correct aujourd'hui, il coûtera cher à l'équipe demain.

17 valeurs écrites en dur

3 entrent dans le calcul

⚠ Vingt-trois lignes seront recopiées dans le prochain test.

⚠ Un paramètre de plus dans un constructeur, et ce test casse.

⚠ Tout semble compter, alors personne n'ose rien changer.

⚠ Un test devrait documenter ; celui-ci oblige à enquêter.

```
[Fact]
public void ShouldEstimateShipmentCO2Emissions() {
    var shipment = new Shipment(
        ShipmentId.Parse("SHP-2026-001872"),
        new Route(
            RouteId.Parse("RTE-8821"),
            Distance.FromKilometers(240.0),
            RouteType.Highway),
        new Parcel(
            ParcelId.Parse("PKG-00152"),
            Weight.FromKilograms(18.0),
            new Dimensions(
                Length.FromMeters(0.6),
                Length.FromMeters(0.4),
                Length.FromMeters(0.3)),
            ParcelCategory.Standard),
        new Vehicle(
            VehicleId.Parse("VEH-271"),
            FuelType.Diesel,
            FuelConsumption.FromLitersPer100Km(6.5),
            EnginePower.FromKilowatts(95.0)),
        new WeatherConditions(
            Temperature.FromCelsius(16.0),
            Humidity.FromPercent(60.0)),
        EmissionFactor.FromKilogramsPerLiter(2.31));

    var calculator = new EmissionCalculator();

    var result = calculator.Estimate(shipment);

    Assert.Equal(36.036, result, precision: 3);
}
```

// 02 · LE PROBLÈME

Un test vert, mais fragile

Correct aujourd'hui, il coûtera cher à l'équipe demain.

- ⚠ Hard-Coded Test Data
- ⚠ Test Code Duplication
- ⚠ Fragile Test
- ⚠ Irrelevant Information
- ⚠ Obscure Test

```
[Fact]
public void ShouldEstimateShipmentCO2Emissions() {
    var shipment = new Shipment(
        ShipmentId.Parse("SHP-2026-001872"),
        new Route(
            RouteId.Parse("RTE-8821"),
            Distance.FromKilometers(240.0),
            RouteType.Highway),
        new Parcel(
            ParcelId.Parse("PKG-00152"),
            Weight.FromKilograms(18.0),
            new Dimensions(
                Length.FromMeters(0.6),
                Length.FromMeters(0.4),
                Length.FromMeters(0.3)),
            ParcelCategory.Standard),
        new Vehicle(
            VehicleId.Parse("VEH-271"),
            FuelType.Diesel,
            FuelConsumption.FromLitersPer100Km(6.5),
            EnginePower.FromKilowatts(95.0)),
        new WeatherConditions(
            Temperature.FromCelsius(16.0),
            Humidity.FromPercent(60.0)),
        EmissionFactor.FromKilogramsPerLiter(2.31));

    var calculator = new EmissionCalculator();

    var result = calculator.Estimate(shipment);

    Assert.Equal(36.036, result, precision: 3);
}
```

// 02 · LE PROBLÈME

Un test vert, mais fragile

Correct aujourd'hui, il coûtera cher à l'équipe demain.

⚠ Hard-Coded Test Data

```
[Fact]
public void ShouldEstimateShipmentCO2Emissions() {
    var shipment = new Shipment(
        ShipmentId.Parse("SHP-2026-001872"),
        new Route(
            RouteId.Parse("RTE-8821"),
            Distance.FromKilometers(240.0),
            RouteType.Highway),
        new Parcel(
            ParcelId.Parse("PKG-00152"),
            Weight.FromKilograms(18.0),
            new Dimensions(
                Length.FromMeters(0.6),
                Length.FromMeters(0.4),
                Length.FromMeters(0.3)),
            ParcelCategory.Standard),
        new Vehicle(
            VehicleId.Parse("VEH-271"),
            FuelType.Diesel,
            FuelConsumption.FromLitersPer100Km(6.5),
            EnginePower.FromKilowatts(95.0)),
        new WeatherConditions(
            Temperature.FromCelsius(16.0),
            Humidity.FromPercent(60.0)),
        EmissionFactor.FromKilogramsPerLiter(2.31));

    var calculator = new EmissionCalculator();

    var result = calculator.Estimate(shipment);

    Assert.Equal(36.036, result, precision: 3);
}
```

// 02 · LE PROBLÈME

Un test vert, mais fragile

Correct aujourd'hui, il coûtera cher à l'équipe demain.

⚠ Test Code Duplication

```
[Fact]
public void ShouldEstimateShipmentCO2Emissions() {
    var shipment = new Shipment(
        ShipmentId.Parse("SHP-2026-001872"),
        new Route(
            RouteId.Parse("RTE-8821"),
            Distance.FromKilometers(240.0),
            RouteType.Highway),
        new Parcel(
            ParcelId.Parse("PKG-00152"),
            Weight.FromKilograms(18.0),
            new Dimensions(
                Length.FromMeters(0.6),
                Length.FromMeters(0.4),
                Length.FromMeters(0.3)),
            ParcelCategory.Standard),
        new Vehicle(
            VehicleId.Parse("VEH-271"),
            FuelType.Diesel,
            FuelConsumption.FromLitersPer100Km(6.5),
            EnginePower.FromKilowatts(95.0)),
        new WeatherConditions(
            Temperature.FromCelsius(16.0),
            Humidity.FromPercent(60.0)),
        EmissionFactor.FromKilogramsPerLiter(2.31));

    var calculator = new EmissionCalculator();

    var result = calculator.Estimate(shipment);

    Assert.Equal(36.036, result, precision: 3);
}
```

// 02 · LE PROBLÈME

Un test vert, mais fragile

Correct aujourd'hui, il coûtera cher à l'équipe demain.

⚠ Fragile Test

```
[Fact]
public void ShouldEstimateShipmentCO2Emissions() {
    var shipment = new Shipment(
        ShipmentId.Parse("SHP-2026-001872"),
        new Route(
            RouteId.Parse("RTE-8821"),
            Distance.FromKilometers(240.0),
            RouteType.Highway),
        new Parcel(
            ParcelId.Parse("PKG-00152"),
            Weight.FromKilograms(18.0),
            new Dimensions(
                Length.FromMeters(0.6),
                Length.FromMeters(0.4),
                Length.FromMeters(0.3)),
            ParcelCategory.Standard),
        new Vehicle(
            VehicleId.Parse("VEH-271"),
            FuelType.Diesel,
            FuelConsumption.FromLitersPer100Km(6.5),
            EnginePower.FromKilowatts(95.0)),
        new WeatherConditions(
            Temperature.FromCelsius(16.0),
            Humidity.FromPercent(60.0)),
        EmissionFactor.FromKilogramsPerLiter(2.31));

    var calculator = new EmissionCalculator();

    var result = calculator.Estimate(shipment);

    Assert.Equal(36.036, result, precision: 3);
}
```

// 02 · LE PROBLÈME

Un test vert, mais fragile

Correct aujourd'hui, il coûtera cher à l'équipe demain.

⚠ Irrelevant Information

```
[Fact]
public void ShouldEstimateShipmentCO2Emissions() {
    var shipment = new Shipment(
        ShipmentId.Parse("SHP-2026-001872"),
        new Route(
            RouteId.Parse("RTE-8821"),
            Distance.FromKilometers(240.0),
            RouteType.Highway),
        new Parcel(
            ParcelId.Parse("PKG-00152"),
            Weight.FromKilograms(18.0),
            new Dimensions(
                Length.FromMeters(0.6),
                Length.FromMeters(0.4),
                Length.FromMeters(0.3)),
            ParcelCategory.Standard),
        new Vehicle(
            VehicleId.Parse("VEH-271"),
            FuelType.Diesel,
            FuelConsumption.FromLitersPer100Km(6.5),
            EnginePower.FromKilowatts(95.0)),
        new WeatherConditions(
            Temperature.FromCelsius(16.0),
            Humidity.FromPercent(60.0)),
        EmissionFactor.FromKilogramsPerLiter(2.31));

    var calculator = new EmissionCalculator();

    var result = calculator.Estimate(shipment);

    Assert.Equal(36.036, result, precision: 3);
}
```

// 02 · LE PROBLÈME

Un test vert, mais fragile

Correct aujourd'hui, il coûtera cher à l'équipe demain.

⚠ Obscure Test

```
[Fact]
public void ShouldEstimateShipmentCO2Emissions() {
    var shipment = new Shipment(
        ShipmentId.Parse("SHP-2026-001872"),
        new Route(
            RouteId.Parse("RTE-8821"),
            Distance.FromKilometers(240.0),
            RouteType.Highway),
        new Parcel(
            ParcelId.Parse("PKG-00152"),
            Weight.FromKilograms(18.0),
            new Dimensions(
                Length.FromMeters(0.6),
                Length.FromMeters(0.4),
                Length.FromMeters(0.3)),
            ParcelCategory.Standard),
        new Vehicle(
            VehicleId.Parse("VEH-271"),
            FuelType.Diesel,
            FuelConsumption.FromLitersPer100Km(6.5),
            EnginePower.FromKilowatts(95.0)),
        new WeatherConditions(
            Temperature.FromCelsius(16.0),
            Humidity.FromPercent(60.0)),
        EmissionFactor.FromKilogramsPerLiter(2.31));

    var calculator = new EmissionCalculator();

    var result = calculator.Estimate(shipment);

    Assert.Equal(36.036, result, precision: 3);
}
```

// 02 · LE PROBLÈME

Un test vert, mais fragile

Correct aujourd'hui, il coûtera cher à l'équipe demain.

- ⚠ Hard-Coded Test Data
- ⚠ Test Code Duplication
- ⚠ Fragile Test
- ⚠ Irrelevant Information
- ⚠ Obscure Test

```
[Fact]
public void ShouldEstimateShipmentCO2Emissions() {
    var shipment = new Shipment(
        ShipmentId.Parse("SHP-2026-001872"),
        new Route(
            RouteId.Parse("RTE-8821"),
            Distance.FromKilometers(240.0),
            RouteType.Highway),
        new Parcel(
            ParcelId.Parse("PKG-00152"),
            Weight.FromKilograms(18.0),
            new Dimensions(
                Length.FromMeters(0.6),
                Length.FromMeters(0.4),
                Length.FromMeters(0.3)),
            ParcelCategory.Standard),
        new Vehicle(
            VehicleId.Parse("VEH-271"),
            FuelType.Diesel,
            FuelConsumption.FromLitersPer100Km(6.5),
            EnginePower.FromKilowatts(95.0)),
        new WeatherConditions(
            Temperature.FromCelsius(16.0),
            Humidity.FromPercent(60.0)),
        EmissionFactor.FromKilogramsPerLiter(2.31));

    var calculator = new EmissionCalculator();

    var result = calculator.Estimate(shipment);

    Assert.Equal(36.036, result, precision: 3);
}
```

Pour notre exemple...

Discutable

Un modèle volontairement imparfait

- Un Shipment fourre-tout
- Des responsabilités potentiellement mal découpées
- Une construction imbriquée sur quatre niveaux

```
[Fact]
public void ShouldEstimateShipmentCO2Emissions() {
    var shipment = new Shipment(
        ShipmentId.Parse("SHP-2026-001872"),
        new Route(
            RouteId.Parse("RTE-8821"),
            Distance.FromKilometers(240.0),
            RouteType.Highway),
        new Parcel(
            ParcelId.Parse("PKG-00152"),
            Weight.FromKilograms(18.0),
            new Dimensions(
                Length.FromMeters(0.6),
                Length.FromMeters(0.4),
                Length.FromMeters(0.3)),
            ParcelCategory.Standard),
        new Vehicle(
            VehicleId.Parse("VEH-271"),
            FuelType.Diesel,
            FuelConsumption.FromLitersPer100Km(6.5),
            EnginePower.FromKilowatts(95.0)),
        new WeatherConditions(
            Temperature.FromCelsius(16.0),
            Humidity.FromPercent(60.0)),
        EmissionFactor.FromKilogramsPerLiter(2.31));

    var calculator = new EmissionCalculator();

    var result = calculator.Estimate(shipment);

    Assert.Equal(36.036, result, precision: 3);
}
```

Pour notre exemple...

Intouchable

Ce que nous ne ferons pas pour faciliter les tests

- Rendre public un constructeur privé
- Ajouter un setter ou exposer un champ
- Créer un objet sans passer par sa factory
- Assouplir une validation métier

Encapsulation maximale

Length.cs

```
public sealed class Length {
    private readonly double _meters;
    private Length(double meters) { _meters = meters; }
    public static Length FromMeters(double meters) {
        ArgumentOutOfRangeException.ThrowIfNegativeOrZero(meters);
        return new Length(meters);
    }
    // …
}
```

```
[Fact]
public void ShouldEstimateShipmentCO2Emissions() {
    var shipment = new Shipment(
        ShipmentId.Parse("SHP-2026-001872"),
        new Route(
            RouteId.Parse("RTE-8821"),
            Distance.FromKilometers(240.0),
            RouteType.Highway),
        new Parcel(
            ParcelId.Parse("PKG-00152"),
            Weight.FromKilograms(18.0),
            new Dimensions(
                Length.FromMeters(0.6),
                Length.FromMeters(0.4),
                Length.FromMeters(0.3)),
            ParcelCategory.Standard),
        new Vehicle(
            VehicleId.Parse("VEH-271"),
            FuelType.Diesel,
            FuelConsumption.FromLitersPer100Km(6.5),
            EnginePower.FromKilowatts(95.0)),
        new WeatherConditions(
            Temperature.FromCelsius(16.0),
            Humidity.FromPercent(60.0)),
        EmissionFactor.FromKilogramsPerLiter(2.31));

    var calculator = new EmissionCalculator();

    var result = calculator.Estimate(shipment);

    Assert.Equal(36.036, result, precision: 3);
}
```

Même démarche avec un modèle ouvert

Notre exemple est le cas le plus strict.

Modèle ouvert

Shipment.cs

```
public class Shipment {
    public string Id { get; set; }
    public double DistanceKm { get; set; }
    public double WeightKg { get; set; }
    // …
}
```

EmissionCalculator.cs

```
public sealed class EmissionCalculator {
    public double Estimate(Shipment shipment) {
        ArgumentOutOfRangeException.ThrowIfNegativeOrZero(shipment.DistanceKm);
        ArgumentOutOfRangeException.ThrowIfNegativeOrZero(shipment.WeightKg);
        // …
    }
}
```

Modèle ouvert — **Encapsulation maximale**

Ce code : setters publics, validation dans le service

Notre exemple : constructeurs privés, factories, Value Objects

Qui peut le plus peut le moins.

Le test de départ

```
var shipment = new Shipment(
    ShipmentId.Parse("SHP-2026-001872"),
    new Route(RouteId.Parse("RTE-8821"), Distance.FromKilometers(240.0), RouteType.Highway),
    new Parcel(ParcelId.Parse("PKG-00152"), Weight.FromKilograms(18.0),
        new Dimensions(Length.FromMeters(0.6), Length.FromMeters(0.4), Length.FromMeters(0.3)),
        ParcelCategory.Standard),
    new Vehicle(VehicleId.Parse("VEH-271"), FuelType.Diesel,
        FuelConsumption.FromLitersPer100Km(6.5), EnginePower.FromKilowatts(95.0)),
    new WeatherConditions(Temperature.FromCelsius(16.0), Humidity.FromPercent(60.0)),
    EmissionFactor.FromKilogramsPerLiter(2.31));

var calculator = new EmissionCalculator();

var result = calculator.Estimate(shipment);

Assert.Equal(36.036, result, precision: 3);
```

- ⚠ Hard-Coded Test Data
- ⚠ Test Code Duplication
- ⚠ Fragile Test
- ⚠ Irrelevant Information
- ⚠ Obscure Test

Nommer ce qui compte rend l'intention explicite

```
// Setup
// prepare dummy arguments
var anyShipmentId = ShipmentId.Parse("SHP-2026-001872");
var anyRouteId = RouteId.Parse("RTE-8821");
var anyRouteType = RouteType.Highway;
var anyParcel = new Parcel(ParcelId.Parse("PKG-00152"), Weight.FromKilograms(18.0),
    new Dimensions(Length.FromMeters(0.6), Length.FromMeters(0.4), Length.FromMeters(0.3)),
    ParcelCategory.Standard);
var anyVehicleId = VehicleId.Parse("VEH-271");
var anyFuelType = FuelType.Diesel;
var anyEnginePower = EnginePower.FromKilowatts(95.0);
var anyWeatherConditions = new WeatherConditions(Temperature.FromCelsius(16.0),
    Humidity.FromPercent(60.0));
// prepare test-specific arguments
var distance = Distance.FromKilometers(240.0);
var consumption = FuelConsumption.FromLitersPer100Km(6.5);
var emissionFactor = EmissionFactor.FromKilogramsPerLiter(2.31);
// prepare arguments depending on test-specific arguments
var anyRouteWithSpecificDistance = new Route(anyRouteId, distance, anyRouteType);
var anyVehicleWithSpecificConsumption = new Vehicle(anyVehicleId, anyFuelType,
    consumption, anyEnginePower);
// assemble the shipment and create the calculator
var shipment = new Shipment(anyShipmentId, anyRouteWithSpecificDistance, anyParcel,
    anyVehicleWithSpecificConsumption, anyWeatherConditions, emissionFactor);
var calculator = new EmissionCalculator();

// Exercise
var result = calculator.Estimate(shipment);

// Verify
Assert.Equal(36.036, result, precision: 3);
```

- ⚠ Hard-Coded Test Data
- ⚠ Test Code Duplication
- ⚠ Fragile Test
- ⚠ Irrelevant Information
- ✓ ~~Obscure Test~~

Des factories sortent une partie du bruit

```
// Setup
// prepare dummy arguments
var anyShipmentId = ShipmentIdFactory.Any();
var anyParcel = ParcelFactory.Any();
var anyWeatherConditions = WeatherConditionsFactory.Any();
// prepare test-specific arguments
var distance = Distance.FromKilometers(240.0);
var consumption = FuelConsumption.FromLitersPer100Km(6.5);
var emissionFactor = EmissionFactor.FromKilogramsPerLiter(2.31);
// prepare arguments depending on test-specific arguments
var anyRouteWithSpecificDistance = RouteFactory.AnyWithSpecificDistance(distance);
var anyVehicleWithSpecificConsumption = VehicleFactory.AnyWithSpecificConsumption(consumption);
// assemble the shipment and create the calculator
var shipment = new Shipment(anyShipmentId, anyRouteWithSpecificDistance, anyParcel,
    anyVehicleWithSpecificConsumption, anyWeatherConditions, emissionFactor);
var calculator = new EmissionCalculator();

// Exercise
var result = calculator.Estimate(shipment);

// Verify
Assert.Equal(36.036, result, precision: 3);
```

- ✓ ~~Hard-Coded Test Data~~
- ⚠ Test Code Duplication
- ⚠ Fragile Test
- ⚠ Irrelevant Information
- ✓ ~~Obscure Test~~

Des factories sortent une partie du bruit

La base : une factory par type, et des factories qui se composent.

TemperatureFactory.cs

```
public static class TemperatureFactory {
    public static Temperature Any() {
        return Temperature.FromCelsius(16.0);
    }
}
```

HumidityFactory.cs

```
public static class HumidityFactory {
    public static Humidity Any() {
        return Humidity.FromPercent(60.0);
    }
}
```

WeatherConditionsFactory.cs

```
public static class WeatherConditionsFactory {
    public static WeatherConditions Any() {
        return new WeatherConditions(
            TemperatureFactory.Any(),
            HumidityFactory.Any());
    }
}
```

Des factories sortent une partie du bruit

La personnalisation : on ne fixe que ce qui compte, le reste vient des factories simples.

RouteFactory.cs

```
public static class RouteFactory {
    public static Route AnyWithSpecificDistance(Distance distance) {
        return new Route(RouteIdFactory.Any(), distance, RouteTypeFactory.Any());
    }
}
```

VehicleFactory.cs

```
public static class VehicleFactory {
    public static Vehicle AnyWithSpecificConsumption(FuelConsumption consumption) {
        return new Vehicle(VehicleIdFactory.Any(), FuelTypeFactory.Any(), consumption, EnginePowerFactory.Any());
    }
}
```

Le builder masque le bruit

```
// Setup
// prepare test-specific arguments
var distance = Distance.FromKilometers(240.0);
var consumption = FuelConsumption.FromLitersPer100Km(6.5);
var emissionFactor = EmissionFactor.FromKilogramsPerLiter(2.31);
// assemble the shipment and create the calculator
var shipment = AnyShipment()
    .WithRoute(anyRoute ⇒ anyRoute.WithDistance(distance))
    .WithVehicle(anyVehicle ⇒ anyVehicle.WithConsumption(consumption))
    .WithEmissionFactor(emissionFactor)
    .Build();
var calculator = new EmissionCalculator();

// Exercise
var result = calculator.Estimate(shipment);

// Verify
Assert.Equal(36.036, result, precision: 3);
```

- ✓ ~~Hard-Coded Test Data~~
- ✓ ~~Test Code Duplication~~
- ✓ ~~Fragile Test~~
- ✓ ~~Irrelevant Information~~
- ✓ ~~Obscure Test~~

Le builder masque le bruit

Un point d'entrée qui dit l'intention : un envoi quelconque.

ShipmentBuilder.cs

```
public static ShipmentBuilder AnyShipment() ⇒ new ShipmentBuilder();
```

Test Data Builder et méthode de création en tête de chaîne : Steve Freeman et Nat Pryce, *Growing Object-Oriented Software, Guided by Tests* (2009).

growing-object-oriented-software.com

Le builder masque le bruit

Ce code suffit déjà pour un premier usage : fournir un "any shipment".

ShipmentBuilder.cs

```
public sealed class ShipmentBuilder {

    private ShipmentId        _id             = ShipmentIdFactory.Any();
    private Route             _route          = RouteFactory.Any();
    private Vehicle           _vehicle        = VehicleFactory.Any();
    private Parcel            _parcel         = ParcelFactory.Any();
    private WeatherConditions _weather        = WeatherConditionsFactory.Any();
    private EmissionFactor    _emissionFactor = EmissionFactorFactory.Any();

    public Shipment Build() {
        return new Shipment(_id, _route, _parcel, _vehicle, _weather, _emissionFactor);
    }

    // …

}
```

Le builder masque le bruit

Ce code ajoute un deuxième usage : fixer une valeur déterminante.

ShipmentBuilder.cs

```
public sealed class ShipmentBuilder {
    // …
    public ShipmentBuilder WithId(ShipmentId id) { _id = id; return this; }
    public ShipmentBuilder WithRoute(Route route) { _route = route; return this; }
    public ShipmentBuilder WithVehicle(Vehicle vehicle) { _vehicle = vehicle; return this; }
    public ShipmentBuilder WithParcel(Parcel parcel) { _parcel = parcel; return this; }
    public ShipmentBuilder WithWeather(WeatherConditions weather) { _weather = weather; return this; }
    public ShipmentBuilder WithEmissionFactor(EmissionFactor emissionFactor) { _emissionFactor = emissionFactor; return this; }
}
```

Le builder masque le bruit

Ce code ajoute un troisième usage : personnaliser une valeur.

ShipmentBuilder.cs

```
public sealed class ShipmentBuilder {

    // …

    public ShipmentBuilder WithRoute(Func<RouteBuilder, RouteBuilder> configure) {
        RouteBuilder configuredRouteBuilder = configure(AnyRoute());
        Route        route                  = configuredRouteBuilder.Build();

        return WithRoute(route);
    }

}
```

Le builder masque le bruit

```
// Setup
var shipment = AnyShipment()
    .WithRoute(anyRoute ⇒ anyRoute.WithDistance(Distance.FromKilometers(240.0)))
    .WithVehicle(anyVehicle ⇒ anyVehicle.WithConsumption(FuelConsumption.FromLitersPer100Km(6.5)))
    .WithEmissionFactor(EmissionFactor.FromKilogramsPerLiter(2.31))
    .Build();
var calculator = new EmissionCalculator();

// Exercise
var result = calculator.Estimate(shipment);

// Verify
Assert.Equal(36.036, result, precision: 3);
```



- ✓ ~~Hard-Coded Test Data~~
- ✓ ~~Test Code Duplication~~
- ✓ ~~Fragile Test~~
- ✓ ~~Irrelevant Information~~
- ✓ ~~Obscure Test~~

Le builder masque le bruit

```
// Setup
var shipment = AnyShipment()
    .WithRoute(anyRoute ⇒ anyRoute.WithDistance(Distance.FromKilometers(240.0)))
    .WithVehicle(anyVehicle ⇒ anyVehicle.WithConsumption(FuelConsumption.FromLitersPer100Km(6.5)))
    .WithEmissionFactor(EmissionFactor.FromKilogramsPerLiter(2.31))
    .Build();
var calculator = new EmissionCalculator();

// Exercise
var result = calculator.Estimate(shipment);

// Verify
Assert.Equal(36.036, result, precision: 3);
```

⚠ Hard-Coded Test Data

✓ ~~Test Code Duplication~~

✓ ~~Fragile Test~~

✓ ~~Irrelevant Information~~

✓ ~~Obscure Test~~

mais le test ment !

Le builder fige des valeurs en dur

Vraiment "any" ?

```
public static ShipmentBuilder AnyShipment() => new ShipmentBuilder();

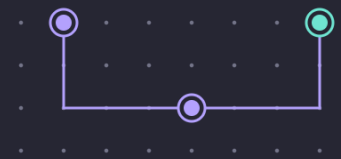
public static class ShipmentIdFactory {
    public static ShipmentId Any() {
        return ShipmentId.Parse("SHP-2026-001872");
    }
}

public static class RouteFactory {
    public static Route Any() {
        return new Route(RouteIdFactory.Any(),
                         Distance.FromKilometers(240.0),
                         RouteTypeFactory.Any());
    }
}

// … and so on for every factory
```

⚠ **Les valeurs du test de départ sont toujours là, cachées dans les factories.**

Un dummy n’a aucune raison d’être figé



FIGÉE

Literal Value

Écrite telle quelle dans le test, comme 240 km.

VARIABLE

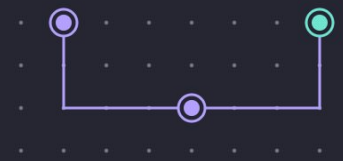
Generated Value

Produite à l’exécution, en séquence ou au hasard.

Nos dummies aujourd’hui

Ce qu’ils pourraient être

Un dummy n’a aucune raison d’être figé



Dans le test : `var anyShipmentId = ShipmentIdFactory.Any();`

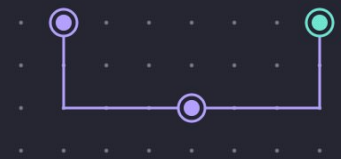
Ce que fait Any() aujourd’hui

```
public static class ShipmentIdFactory {
    public static ShipmentId Any() {
        return ShipmentId.Parse("SHP-2026-001872");
    }
}
```

Ce qu’on imagine en lisant Any()

```
lic static class ShipmentIdFactory {
 public static ShipmentId Any() {
     // generate random string
     return ShipmentId.Parse(randomString);
 }
```

Un dummy n’a aucune raison d’être figé



Dans le test : `var anyShipmentId = ShipmentIdFactory.Any();`

Ce que fait Any() aujourd'hui

```
public static class ShipmentIdFactory {
    public static ShipmentId Any() {
        return ShipmentId.Parse("SHP-2026-001872");
    }
}
```

Ce qu’on imagine en lisant Any()

```
public static class ShipmentIdFactory {
    public static ShipmentId Any() {
        // generate random string
        return ShipmentId.Parse(randomString);
    }
}
```

Oui, mais voilà le problème

```
public sealed class ShipmentId {

    internal static readonly Regex Format = new(@"^SHP-\d{4}-\d{6}$");

    public string Value { get; }

    private ShipmentId(string value) {
        Value = value;
    }

    public static ShipmentId Parse(string value) {
        if (!Format.IsMatch(value)) {
            throw new FormatException($"Invalid shipment id: '{value}'.");
        }
        return new ShipmentId(value);
    }
}
```

⚠ Une chaîne tirée au hasard ne respecte pas ce format : Parse la refuse.

Revenons à ce qu'on voudrait…

Une valeur différente à chaque exécution, toujours valide.

ShipmentTests.cs

```
var anyShipmentId = ShipmentIdFactory.Any();
```

ShipmentIdFactory.cs

```
public static class ShipmentIdFactory {
    public static ShipmentId Any() {
        return Any.StringMatching(ShipmentId.Format)
                  .As(ShipmentId.Parse)
                  .Generate();
    }
}
```

Pas mal, non ?

JustDummies

Un DSL fluide pour générer des valeurs de test arbitraires mais valides : des dummies.



```
string reference = Any.String()
                       .AlphaNumeric()
                       .InUpperCase()
                       .StartingWith("ORD-")
                       .WithLengthBetween(8, 20)
                       .Generate();

// → produced ORD-4KMVJ2EIUA
// → produced ORD-JKOGOR1UU6M5
// → produced ORD-YB3Z4AK4164U8LA
```

Autres options

	JustDummies	Bogus	AutoFixture
Produire des valeurs qui respectent les règles du domaine	Fait pour ça	Avec effort	Avec effort
Construire automatiquement tout un graphe d'objets	Avec effort[a]	Avec effort	Fait pour ça
Produire des données crédibles (noms, e-mails, adresses…)	Avec effort[b]	Fait pour ça	Pas son rôle
Rejouer à l'identique une exécution qui a échoué	Fait pour ça	Avec effort	Pas son rôle
Signaler les erreurs d'usage dès la compilation	Fait pour ça	Payant	Pas son rôle

[a] Pour un petit graphe comme celui de notre exemple, any - le CLI scaffolder de JustDummies - génère un générateur en une commande.

[b] En lui fournissant sa propre liste de valeurs :

```
string[] firstNames   = File.ReadAllLines("first-names.txt");
string   anyFirstName = Any.ElementOf(firstNames).Generate();
```

Autres options

Vous avez entendu parler du property-based testing ?

```
[Property]
public Property DivideUndoesMultiply() {
    var numbers  = Arb.Generate<double>().ToArbitrary();
    var divisors = Arb.Generate<double>()
                      .Where(b ⇒ b ≠ 0)
                      .ToArbitrary();

    return Prop.ForAll(numbers, divisors, (a, b) ⇒ {
        // Setup
        var calculator = new Calculator();

        // Exercise
        var product = calculator.Multiply(a, b);
        var result  = calculator.Divide(product, b);

        // Verify
        return result == a;
    });
}
```



Autres options

Vous avez entendu parler du property-based testing ?

```
[Property]
public Property DivideUndoesMultiply() {
    var numbers  = Arb.Generate<double>().ToArbitrary();
    var divisors = Arb.Generate<double>()
                      .Where(b ⇒ b ≠ 0)
                      .ToArbitrary();

    return Prop.ForAll(numbers, divisors, (a, b) ⇒ {
        // Setup
        var calculator = new Calculator();

        // Exercise
        var product = calculator.Multiply(a, b);
        var result  = calculator.Divide(product, b);

        // Verify
        return result == a;
    });
}
```

Échoue dès le premier run !

FsCheck · CsCheck

Property-based testing

- Attaquer une règle jusqu'à la casser
- Des centaines de cas à chaque exécution
- Des valeurs biaisées vers les cas limites

Autres options

JustDummies, lui, montre qu'une valeur ne compte pas.

Ou une autre bibliothèque qui vous convient mieux…

JustDummies

Dummies

- Montrer que la valeur ne compte pas
- Un seul tirage par exécution
- Des valeurs quelconques, mais valides
- Un échec rejoué à l'identique

Chaque valeur figée devient un invariant déclaré

FIGÉ DANS LE BUILDER	DÉCLARÉ AVEC JUSTDUMMIES	L'INVARIANT DU DOMAINE
`ShipmentId.Parse("SHP-2026-001872")`	`Any.StringMatching(@"SHP-\d{4}-\d{6}").As(ShipmentId.Parse)`	Un identifiant respecte son format
`Weight.FromKilograms(18.0)`	`Any.Double().Positive().As(Weight.FromKilograms)`	Un poids est strictement positif
`EnginePower.FromKilowatts(95.0)`	`Any.Double().Positive().As(EnginePower.FromKilowatts)`	Une puissance aussi
`Temperature.FromCelsius(16.0)`	`Any.Double().GreaterThan(-273.15).As(Temperature.FromCelsius)`	Rien sous le zéro absolu
`Humidity.FromPercent(60.0)`	`Any.Double().Between(0, 100).As(Humidity.FromPercent)`	Une humidité tient entre 0 et 100 %
`RouteType.Highway`	`Any.Enum<RouteType>()`	Toute valeur déclarée du type

Générateur (scaffolding)

```
$ any generate Shipment --entry-point any
```

dum lit les gardes des constructeurs et écrit un premier jet que vous possédez. Ce qu'il ne sait pas inférer devient un TODO qui casse le build.

AnyShipment.cs · écrit par dum

```
public sealed partial class AnyShipment : IAny<Shipment> {
    public AnyShipment WithRoute(Action<AnyRoute> configure) { … }
    public AnyShipment WithVehicle(Action<AnyVehicle> configure) { … }
    public AnyShipment WithEmissionFactor(EmissionFactor value) { … }
    // … WithId, WithParcel, WithWeather
    public Shipment Generate() { … }
}
```

AnyShipment.Entry.cs · --entry-point any

```
public static class AnyShipmentEntry {
    extension(Any) {
        public static AnyShipment Shipment() { … }
    }
}
```

Le même test, qui ne ment plus

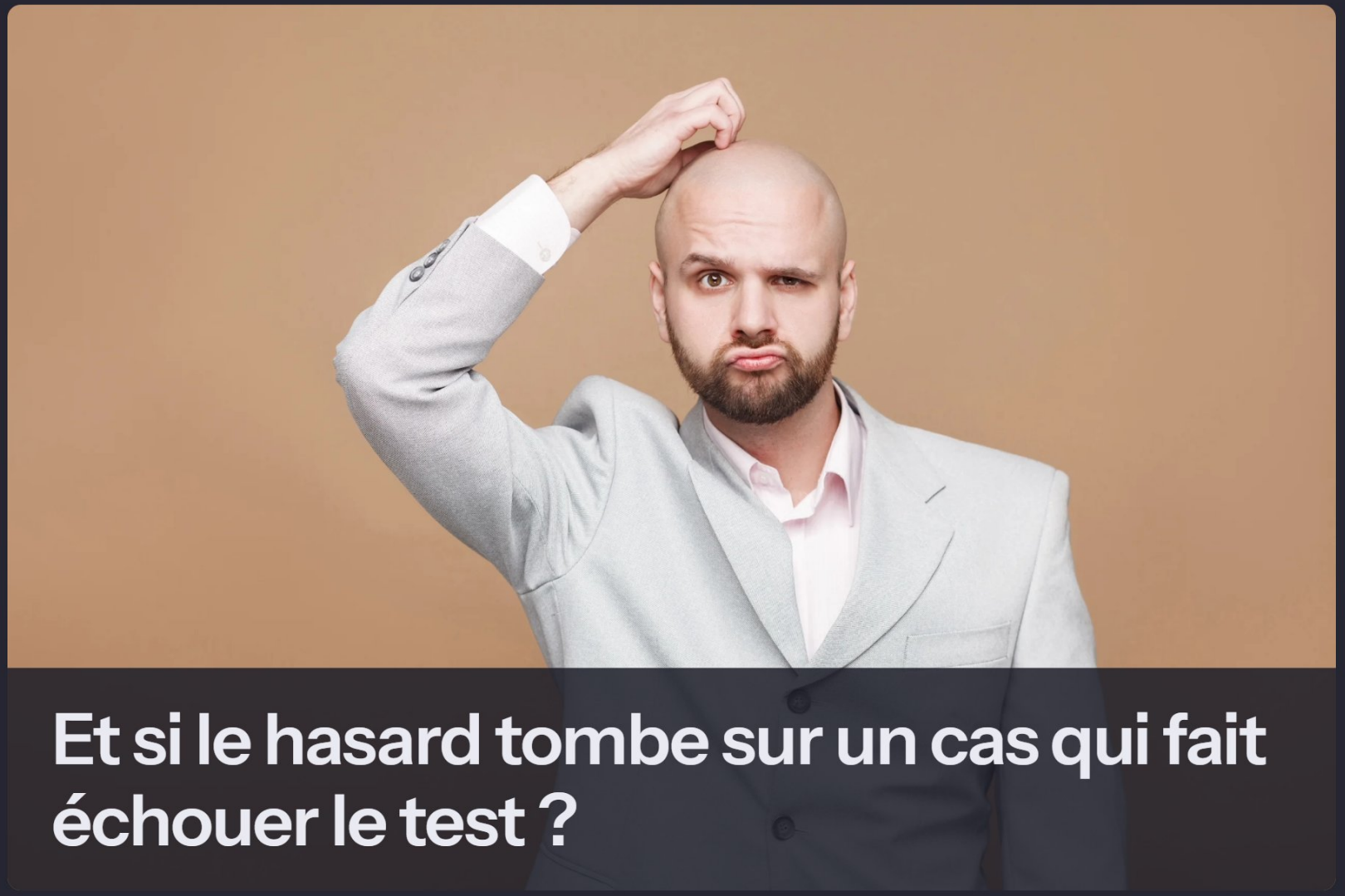
```
[Fact]
public void ShouldEstimateShipmentCO2Emissions() {
    // Setup
    var shipment = Any.Shipment()
        .WithRoute(anyRoute ⇒ anyRoute.WithDistance(Distance.FromKilometers(240.0)))
        .WithVehicle(anyVehicle ⇒ anyVehicle.WithConsumption(FuelConsumption.FromLitersPer100Km(6.5)))
        .WithEmissionFactor(EmissionFactor.FromKilogramsPerLiter(2.31))
        .Generate();
    var calculator = new EmissionCalculator();

    // Exercise
    var result = calculator.Estimate(shipment);

    // Verify
    Assert.Equal(36.036, result, precision: 3);
}
```



- ✓ ~~Hard-Coded Test Data~~
- ✓ ~~Test Code Duplication~~
- ✓ ~~Fragile Test~~
- ✓ ~~Irrelevant Information~~
- ✓ ~~Obscure Test~~


Et si le hasard tombe sur un cas qui fait échouer le test ?


Tant mieux :
on a peut-être détecté un bug !


D’accord, mais comment reproduire un échec tiré au hasard ?

// 05 · REJOUER

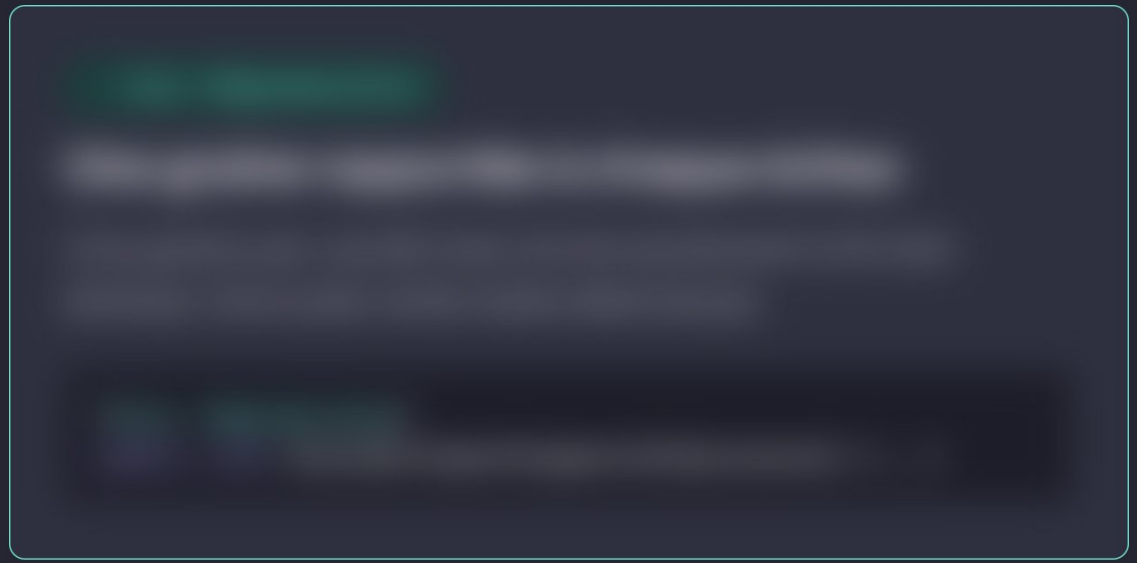
Un échec n’a de valeur que s’il se rejoue

Sans graine

Rouge sur la CI, vert à la relance

Personne ne sait quelles valeurs ont échoué. Un test qui varie sans pouvoir rejouer est un test flaky.

```
[Fact]
public void ShouldEstimateShipmentCO2Emissions() { … }
```



// 05 · REJOUER

Un échec n’a de valeur que s’il se rejoue

Sans graine

Rouge sur la CI, vert à la relance

Personne ne sait quelles valeurs ont échoué. Un test qui varie sans pouvoir rejouer est un test flaky.

```
[Fact]
public void ShouldEstimateShipmentCO2Emissions() { … }
```

✓ Avec [Reproducible]

Une graine rapportée à chaque échec

Une graine par cas de test, écrite seulement si le test échoue. Une suite verte reste silencieuse.

```
[Fact, Reproducible]
public void ShouldEstimateShipmentCO2Emissions() { … }
```

Sans xUnit v3 : Any.Reproducibly(() => { … }) enveloppe le corps du test et rapporte la graine de la même façon.

// 05 · REJOUER

Épingler la graine, corriger, puis la retirer

01 · ROUGE

Le test passe au rouge

→ → → → →

Sortie du test

```
[FAIL] ShipmentTests.ShouldEstimateShipmentCO2Emissions
  Assert.Equal() Failure: Values are not within 3 decimal places
  Expected: 36.036
  Actual: 39.64 (rounded from 39.6396)
```

// 05 · REJOUER

Épingler la graine, corriger, puis la retirer

01 · ROUGE

Le test passe au rouge

→

02

Lire la graine rapportée

→ → → →

Sortie du test

```
[FAIL] [illegible]
  Assert.Equal() Failure: Values are not within 3 decimal places
  Expected: 36.036
  Actual:   39.04 (rounded from 39.0396)

[JustDummies] These arbitrary values were seeded with 1743029518.
Reproduce this run with [Reproducible(Seed = 1743029518)].
```

// 05 · REJOUER

Épingler la graine, corriger, puis la retirer

01 · ROUGE
Le test passe au rouge

→

02
Lire la graine rapportée

→

03
L'épingler sur le test

→ → →

ShipmentTests.cs

```
[Fact, Reproducible(Seed = 1743029518)]
public void ShouldEstimateShipmentCO2Emissions() { … }
```

// 05 · REJOUER

Épingler la graine, corriger, puis la retirer

01 · ROUGE
Le test passe au rouge

→

02
Lire la graine rapportée

→

03
L'épingler sur le test

→

04
Déboguer sur les valeurs exactes

→



→



// 05 · REJOUER

Épingler la graine, corriger, puis la retirer

01 · ROUGE
Le test passe au rouge

→

02
Lire la graine rapportée

→

03
L'épingler sur le test

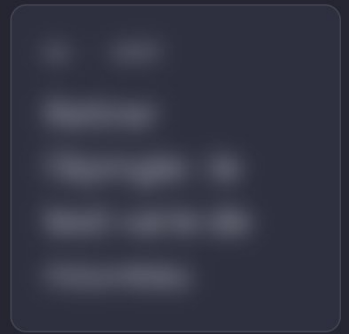
→

04
Déboguer sur les valeurs exactes

→

05
Corriger le défaut

→



// 05 · REJOUER

Épingler la graine, corriger, puis la retirer

01 · ROUGE
Le test passe au rouge

→

02
Lire la graine rapportée

→

03
L'épingler sur le test

→

04
Déboguer sur les valeurs exactes

→

05
Corriger le défaut

→

06 · VERT
Retirer l'épingle : le test varie de nouveau

ShipmentTests.cs

```
[Fact, Reproducible]
public void ShouldEstimateShipmentCO2Emissions() { … }
```

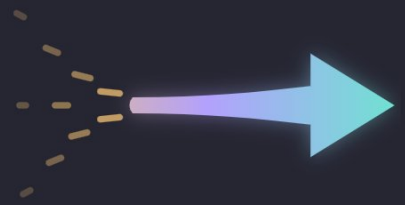
Avant / Après

```
[Fact]
public void ShouldEstimateShipmentCO2Emissions() {
    var shipment = new Shipment(
        ShipmentId.Parse("SHP-2026-001872"),
        new Route(
            RouteId.Parse("RTE-8821"),
            Distance.FromKilometers(240.0),
            RouteType.Highway),
        new Parcel(
            ParcelId.Parse("PKG-00152"),
            Weight.FromKilograms(18.0),
            new Dimensions(
                Length.FromMeters(0.6),
                Length.FromMeters(0.4),
                Length.FromMeters(0.3)),
            ParcelCategory.Standard),
        new Vehicle(
            VehicleId.Parse("VEH-271"),
            FuelType.Diesel,
            FuelConsumption.FromLitersPer100Km(6.5),
            EnginePower.FromKilowatts(95.0)),
        new WeatherConditions(
            Temperature.FromCelsius(16.0),
            Humidity.FromPercent(60.0)),
        EmissionFactor.FromKilogramsPerLiter(2.31));

    var calculator = new EmissionCalculator();

    var result = calculator.Estimate(shipment);

    Assert.Equal(36.036, result, precision: 3);
}
```



```
[Fact, Reproducible]
public void ShouldEstimateShipmentCO2Emissions() {
    // Setup
    var shipment = Any.Shipment()
        .WithRoute(anyRoute ⇒ anyRoute
            .WithDistance(Distance.FromKilometers(240.0)))
        .WithVehicle(anyVehicle ⇒ anyVehicle
            .WithConsumption(FuelConsumption.FromLitersPer100Km(6.5)))
        .WithEmissionFactor(EmissionFactor.FromKilogramsPerLiter(2.31))
        .Generate();
    var calculator = new EmissionCalculator();

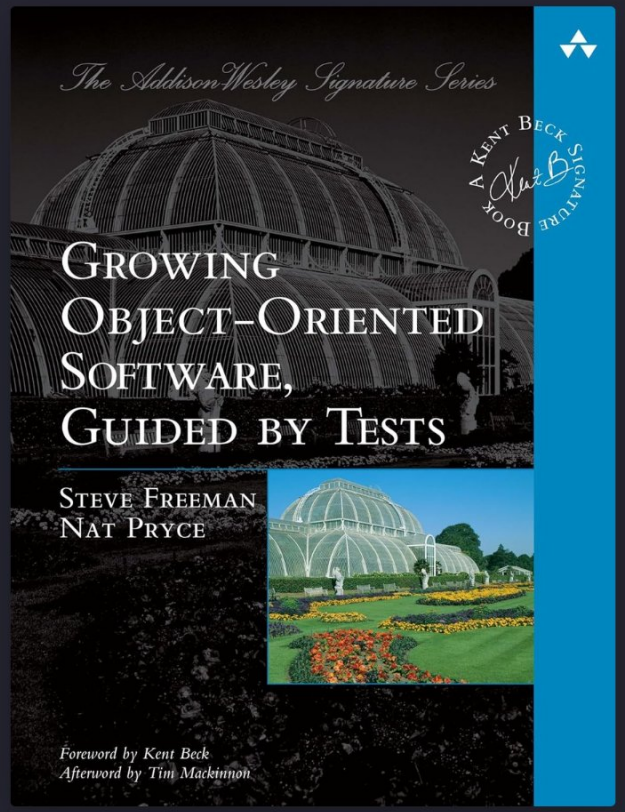
    // Exercise
    var result = calculator.Estimate(shipment);

    // Verify
    Assert.Equal(36.036, result, precision: 3);
}
```

Bibliographie

Où retrouver les références de cette présentation

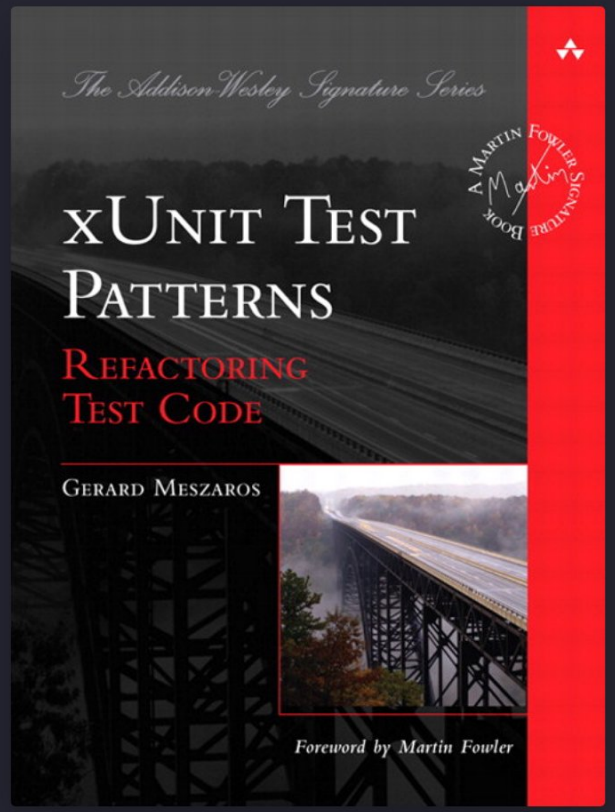
growing-object-oriented-software.com



Bibliographie

Où retrouver les références de cette présentation

xunitpatterns.com



// 06 · BILAN

Rien n'a été inventé ici

Les défauts corrigés sont décrits par Gerard Meszaros.

xunitpatterns.com

Smells corrigés

Obscure Test · Irrelevant Information	Le test noyé dans des détails qui ne comptent pas	16–17
Hard-Coded Test Data	Des valeurs figées dans le test, puis dans le builder	16, 40
Test Code Duplication	Vingt-trois lignes à recopier dans chaque nouveau test	17
Fragile Test · Interface Sensitivity	Un paramètre de plus dans un constructeur, et le test casse	17
Erratic Test · Nondeterministic Test	Un test qui varie sans pouvoir rejouer ses échecs	54–58

// 06 · BILAN

Rien n'a été inventé ici

Les techniques employées viennent de Meszaros, de Nat Pryce et de ThoughtWorks.

xunitpatterns.com

✓ Patterns appliqués

Four-Phase Test	Setup, Exercise, Verify : préparer, exécuter, vérifier	29–53
In-line Setup	Toute la préparation écrite dans le test	16
Delegated Setup	La préparation confiée à des factories, puis à un builder	30–40
Anonymous Creation Method	Any() : une valeur dont le test ne se soucie pas	30, 42
Parameterized Anonymous Creation Method	On ne fixe que ce qui compte, tout le reste est anonyme	30–38, 53
Literal Value	Les trois valeurs déterminantes, écrites en clair	38, 53
Derived Value	Calculée à partir d'autres valeurs : présentée, non utilisée	15
Dummy Attribute	Variante du Dummy Object : les quatorze autres valeurs	17, 29–53
Random Generated Value	Des valeurs tirées au hasard, la graine rapportée à chaque échec	45–64

Ailleurs que chez Meszaros

Object Mother	Des factories de test, nées chez ThoughtWorks	30–32
Test Data Builder	Un builder pour les données de test, de Nat Pryce	33–40

// 06 · BILAN

Les trois apports de JustDummies

Le refactoring a rendu le test lisible. JustDummies l'a rendu honnête.



justdummies.io

```
Any.StringMatching(ShipmentId.Format)
```

Des dummies qui varient

Des valeurs valides qui changent à chaque exécution et montrent que le test n'en dépend pas.

slides 45–51

```
$ dum generate Shipment
```

Des builders générés

Pas besoin d'écrire des factories ou des builders à partir de zéro : le générateur aide à construire les objets du test.

slides 52–53

```
[Fact, Reproducible]
```

Des échecs rejouables

Chaque échec se rejoue à l'identique : le test varie sans jamais devenir flaky.

slides 57–64

Constat



0

classe métier modifiée

Shipment, Route, Parcel, Vehicle restent tels quels, défauts compris.

0

propriété exposée pour les tests

L'encapsulation forte est intacte.

0

factory ou validation contournée

Chaque Value Object passe par sa vraie factory, y compris les dummies.


#messy


#dogmatic


#pragmatic

// À RETENIR · 1/3

Montrer ce qui compte, et seulement ce qui compte, rend un test explicite.

Des valeurs clés visibles, et aucune valeur parasite.

// À RETENIR · 2/3

Faire varier ce qui ne compte pas donne un test qui ne ment plus.

Et quand un test échoue, soit il est mal écrit, soit il a trouvé un bug.

// À RETENIR · 3/3

Ce n’est pas au modèle métier de se dégrader pour faciliter les tests.

C’est à notre façon de préparer les données de test de s’adapter aux exigences du modèle.

// FIN

Merci.
Questions ?



// Voir aussi : **Bogus** · **AutoFixture**

justdummies.io

xunitpatterns.com

growing-object-oriented-software.com